AWS Invoice Pipeline — Deployment Guide

> *Step-by-step instructions for deploying the full AWS Invoice Processing Pipeline into your AWS account — backend infrastructure, Lambda functions, and React frontend.*

Table of Contents

🛠️ Prerequisites

Before you begin, ensure the following are installed and configured:

Tool	Version	Purpose
Node.js	v20+	Runtime for Lambda builds and frontend
PNPM	Latest	Package manager (`npm install -g pnpm`)
AWS CLI	v2+	AWS account access (`aws configure`)
AWS SAM CLI	v1.100+	Serverless Application Model builds and deploys
Git	Latest	Version control

AWS Account Requirements:

- An IAM user/role with **AdministratorAccess** (or equivalent scoped permissions for S3, DynamoDB, Lambda, API Gateway, Step Functions, SES, SNS, Amplify, Secrets Manager, Bedrock, Textract)
- AWS CLI configured with valid credentials (`aws sts get-caller-identity` should return your account info)

🏗️ 1. Preparing the Backend

The backend consists of 5 Lambda functions and a shared library, all written in TypeScript. Dependencies must be installed before building.

From the **project root**:

```
# Install dependencies for all 5 Lambdas + shared code + frontend
make install
```

This runs `pnpm install` in each of the following directories:

- `backend/lambdas/approval-handler/`
- `backend/lambdas/audit-logger/`
- `backend/lambdas/bedrock-validator/`
- `backend/lambdas/invoice-ingestion/`
- `backend/lambdas/textract-processor/`
- `frontend/`

🔐 2. Configuring Secrets and Environment

2.1 — Bedrock Model Access

Ensure your AWS account has access to the AI model used for validation and fallback extraction.

1. Navigate to the **Amazon Bedrock** console → **Model access** (sidebar).
2. Request access to the model specified in the `BedrockModelId` SAM parameter.
   - **Default model**: `global.anthropic.claude-sonnet-4-6`
   - The model is used by both the `textract-processor` (fallback extraction) and `bedrock-validator` (AI validation).
3. Wait for the "Access granted" status before deploying.

> ***Note**: If Textract's `AnalyzeExpense` API is not available in your account (e.g., `SubscriptionRequiredException`), the textract-processor Lambda automatically falls back to the Bedrock Converse API for document extraction. Ensure your Bedrock model supports document/image input.*

2.2 — SES Email Verification

The pipeline sends and receives emails via Amazon SES.

1. Navigate to **Amazon SES** console → **Verified identities**.
2. Create a new identity for your **sender email** (e.g., `admin@yourdomain.com`).
3. Click the verification link sent to that email.
4. If your SES account is in **sandbox mode** (the default for new accounts):
   - You must also verify the **reviewer email** address (the recipient of approval notifications).
   - To send to unverified addresses, request production access from the SES console.

> ***Sandbox mode warning**: In sandbox mode, SES can only send emails to verified addresses. The pipeline handles SES failures gracefully — approval emails may not deliver, but the dashboard UI approval flow will still work.*

🚀 3. Building and Deploying Infrastructure (AWS SAM)

The entire backend infrastructure is defined in `infrastructure/template.yaml` as an AWS SAM template.

Step 3.1 — Build the Application

SAM uses `esbuild` to compile TypeScript Lambdas:

```
make build
# Runs: sam build -t infrastructure/template.yaml --build-in-source
```

Step 3.2 — Deploy to AWS (First Time)

For the **first deployment**, use guided mode to configure parameters:

```
make deploy-guided
# Runs: sam deploy --guided -t .aws-sam/build/template.yaml
```

You will be prompted for the following parameters:

Parameter	Description	Default
Stack Name	CloudFormation stack name	`aws-invoice-pipeline`
AWS Region	Deployment region	`ap-south-1`
Environment	Stage name (`dev` / `staging` / `prod`)	`prod`
ReviewerEmail	Email for exception approval notifications	—
SenderEmail	SES-verified sender email address	—
BedrockModelId	Bedrock model for AI validation/extraction	`global.anthropic.claude-sonnet-4-6`
ConfidenceThreshold	Minimum confidence for auto-approval (0–100)	`85`
Allow SAM CLI IAM role creation	Required for Lambda execution roles	`Y`
Disable rollback	Recommended for debugging first deploy	`Y`

After the guided deploy succeeds, SAM saves your configuration to `samconfig.toml`.

Step 3.3 — Subsequent Deployments

```
make deploy
```

Step 3.4 — Note the Outputs

Once deployed, SAM outputs critical values. Save these:

Output	Description	Example
`ApiEndpoint`	API Gateway base URL	`https://xyz.execute-api.ap-south-1.amazonaws.com/prod/`

`AmplifyAppId`	Amplify App ID for frontend hosting	`d1234567abcde`
`AmplifyURL`	Public frontend URL	`https://main.d1234567abcde.amplifyapp.com`
`RawBucketName`	S3 bucket for raw invoice PDFs	`invoice-pipeline-raw-123456789012`
`AuditBucketName`	S3 bucket for audit trail storage	`invoice-pipeline-audit-123456789012`
`StateMachineArn`	Step Functions state machine ARN	`arn:aws:states:...`
`InvoiceTableName`	DynamoDB invoice records table	`InvoiceRecords-prod`
`AuditTableName`	DynamoDB audit entries table	`AuditEntries-prod`
`SNSTopicArn`	SNS notification topic	`arn:aws:sns:...`

💻 4. Deploying the React Frontend

The React dashboard (`frontend/`) is deployed using **AWS Amplify** hosting.

Step 4.1 — Set the API URL

Create a `.env` file in the `frontend/` directory with the API Gateway endpoint from step 3.4:

```
VITE_API_URL=https://<your-api-id>.execute-api.<region>.amazonaws.com/prod
```

Step 4.2 — Deploy to Amplify

Run the deployment script from the **project root**:

```
./deploy_frontend.sh
```

This script:

1. Builds the React frontend (`pnpm run build`)
2. Creates a deployment ZIP
3. Uploads to Amplify via the AWS CLI
4. Outputs the **Amplify Public URL** (e.g., `https://main.<app-id>.amplifyapp.com`)

Step 4.3 — Verify Frontend Access

Open the Amplify URL in your browser. You should see the Invoice Pipeline dashboard.

⚙️ 5. Post-Deployment Configuration

5.1 — Confirm SNS Subscription

After deployment, the reviewer email receives an **SNS subscription confirmation email**. Click **"Confirm subscription"** to start receiving pipeline notifications.

5.2 — Verify SES Email Templates

If approval emails aren't being delivered:

1. Check the SES console → **Sending Statistics** for bounce/complaint metrics.
2. Ensure the sender email address matches the `SenderEmail` parameter.
3. If in sandbox mode, verify the reviewer email in SES.

5.3 — Test the Pipeline

Upload a sample invoice PDF via the React dashboard to verify the full pipeline executes correctly. Monitor progress in the AWS Step Functions console.

🧪 6. Verification

Step	Action	Expected Outcome
1	Visit the Amplify frontend URL	Dashboard loads with 0 invoices
2	Navigate to **Upload Invoice**	Drag-and-drop zone and form appear
3	Upload a sample PDF	Invoice appears with status `IN_PROGRESS`
4	Check **Dashboard** / **Invoices** tab	Invoice progresses from `IN_PROGRESS` → `PROCESSED`
5	Upload an invoice with anomalies	Status becomes `EXCEPTION` or `IN_REVIEW`
6	Check reviewer email inbox	Approval email with Approve/Reject links arrives
7	Click **Approve** in the email	HTML confirmation page appears
8	Check invoice status in dashboard	Status updated to `PROCESSED`

📊 7. Monitoring & Observability

CloudWatch Logs

Each Lambda function writes structured logs to CloudWatch Logs:

Log Group	Lambda
`/aws/lambda/invoice-ingestion-{env}`	Invoice ingestion
`/aws/lambda/textract-processor-{env}`	Textract extraction
`/aws/lambda/bedrock-validator-{env}`	Bedrock validation
`/aws/lambda/approval-handler-{env}`	Approvals & API endpoints
`/aws/lambda/audit-logger-{env}`	Audit log persistence

Step Functions Console

Monitor pipeline executions in the **Step Functions** console:

- Navigate to **State machines** → `InvoicePipelineStateMachine-{env}`
- View execution history, input/output payloads, and error details
- The state machine has built-in retry logic with exponential backoff

Key Metrics to Monitor

Metric	Where to Find	Alert Threshold
Lambda errors	CloudWatch → Lambda → Errors	> 0 errors/min
Lambda duration	CloudWatch → Lambda → Duration	> 80% of timeout
DynamoDB throttling	CloudWatch → DynamoDB → ThrottledRequests	> 0
Step Functions failures	Step Functions → Executions → Failed	> 0
SES bounce rate	SES → Sending Statistics	> 5%

💰 8. Cost Estimation

All resources use **pay-per-use** pricing. Estimated monthly costs for **100 invoices/month**:

Service	Usage	Estimated Cost
Lambda	~500 invocations, avg 5s each	~$0.05
DynamoDB	On-demand, ~200 writes + 1000 reads	~$0.10
S3	~500 MB stored (PDFs + audit JSONs)	~$0.01
Textract	AnalyzeExpense: 100 pages	~$1.00
Bedrock	Claude validation: ~100 calls	~$0.50
Step Functions	100 executions, ~6 transitions each	~$0.02
API Gateway	~2000 REST API calls	~$0.01
SES	~200 emails	~$0.02
Amplify	Static hosting + builds	~$0.00 (free tier)
Total		**~$1.71/month**

> ***Note***: *Costs scale linearly with invoice volume. Textract and Bedrock are the primary cost drivers at high volumes.*

🔒 9. Security Best Practices

Practice	Status	Detail

No hardcoded credentials	✅	All credentials managed via IAM roles and Secrets Manager
S3 bucket encryption	✅	AES-256 server-side encryption on both buckets
S3 public access blocked	✅	`BlockPublicAcls`, `BlockPublicPolicy`, `IgnorePublicAcls`, `RestrictPublicBuckets` enabled
DynamoDB PITR	✅	Point-in-Time Recovery enabled for both tables
IAM least privilege	✅	Lambda roles use SAM policy templates (e.g., `S3ReadPolicy`, `DynamoDBCrudPolicy`)
SES sender restriction	✅	SES permissions conditioned on `ses:FromAddress`
CORS restriction	⚠️	Currently allows all origins (`*`). **Restrict to your Amplify domain in production.**
API authentication	⚠️	No API key or IAM auth on API Gateway. Consider adding Cognito or API keys for production.
Audit retention	✅	S3 audit bucket: 90-day transition to Glacier, 7-year retention

---

🔧 10. Troubleshooting

Common Deployment Errors

Error	Cause	Fix
`S3 bucket already exists`	Bucket name collision (globally unique)	The template uses `{accountId}` suffix — ensure no existing bucket with that name
`SubscriptionRequiredException` (Textract)	Textract not enabled	The Lambda auto-falls back to Bedrock extraction. No action needed unless you want Textract.
`AccessDeniedException` (Bedrock)	Model access not granted	Request model access in Bedrock console → Model access
`CREATE_FAILED` on `RawBucket`	S3 event notification circular dependency	SAM handles this automatically. Retry the deployment.
`Email address is not verified` (SES)	Sender email not verified in SES	Verify email in SES → Verified identities
`Function not found` during `sam deploy`	`sam build` not run	Run `make build` before `make deploy`
`Template format error`	SAM CLI version too old	Upgrade SAM CLI: `pip install --upgrade aws-sam-cli`

Common Runtime Errors

Error	Where	Fix
`STATE_MACHINE_ARN not set`	Ingestion Lambda logs	Check the `STATE_MACHINE_ARN` env var in the Lambda configuration
`No extraction data provided`	Bedrock validator logs	Textract step failed — check textract-processor logs
Approval emails not received	SES	Check SES sandbox mode, verify both sender and recipient emails
Invoice stuck in `IN_PROGRESS`	Step Functions	Check execution history for failed states

---

🧹 11. Cleanup

To avoid ongoing AWS charges, delete all resources:

Step 1 — Empty S3 Buckets

CloudFormation cannot delete non-empty S3 buckets. Empty them first:

```
cd backend/lambdas/textract-processor
node wipe_data.mjs
```

Or manually via AWS CLI:

```
aws s3 rm s3://invoice-pipeline-raw-<account-id> --recursive
aws s3 rm s3://invoice-pipeline-audit-<account-id> --recursive
```

Step 2 — Delete the SAM Stack

```
sam delete --stack-name aws-invoice-pipeline
```

Step 3 — Delete the Amplify App

Delete from the **AWS Amplify console** → Select app → **Actions** → **Delete app**.

📋 Environment Variables Reference

Environment variables set on all Lambda functions via SAM Globals:

Variable	Description	Source
`ENVIRONMENT`	Deployment environment (`dev/staging/prod`)	SAM `Environment` parameter
`INVOICE_TABLE`	DynamoDB invoice table name	SAM `InvoiceTable` resource ref
`AUDIT_TABLE`	DynamoDB audit table name	SAM `AuditTable` resource ref

RAW_BUCKET	S3 bucket for raw PDFs	Constructed: invoice-pipeline-raw-{accountId}
AUDIT_BUCKET	S3 bucket for audit trail	SAM AuditBucket resource ref
SNS_TOPIC_ARN	SNS notification topic ARN	SAM PipelineNotificationTopic ref
SES_SENDER_EMAIL	Verified SES sender email	SAM SenderEmail parameter

Lambda-specific variables:

Lambda	Variable	Description
invoice-ingestion	STATE_MACHINE_ARN	Step Functions state machine ARN
textract-processor	BEDROCK_MODEL_ID	Bedrock model for fallback extraction
bedrock-validator	BEDROCK_MODEL_ID	Bedrock model for AI validation
bedrock-validator	CONFIDENCE_THRESHOLD	Min confidence for auto-approval (default: 85)
approval-handler	APPROVAL_API_URL	API Gateway base URL (auto-discovered if placeholder)
approval-handler	REVIEWER_EMAIL	Email for approval notifications

---

📤 SAM Template Outputs

All outputs from the deployed CloudFormation stack:

Output Key	Description
AmplifyAppId	AWS Amplify Application ID
AmplifyURL	Public URL of the frontend dashboard
ApiEndpoint	API Gateway endpoint URL
StateMachineArn	Step Functions state machine ARN
RawBucketName	S3 bucket for raw invoice PDFs
AuditBucketName	S3 bucket for audit trail
InvoiceTableName	DynamoDB invoice records table
AuditTableName	DynamoDB audit entries table
SNSTopicArn	SNS notification topic ARN
InvoiceIngestionFunctionArn	Invoice ingestion Lambda ARN
TextractProcessorFunctionArn	Textract processor Lambda ARN
BedrockValidatorFunctionArn	Bedrock validator Lambda ARN
ApprovalHandlerFunctionArn	Approval handler Lambda ARN

`AuditLoggerFunctionArn`	Audit logger Lambda ARN